COMP 203

Python Project

TESLA STOCK MARKET

Predicts Tesla's stock price using historical data and analysis

By:

Farah Balobaid
Aljawharah Alzuabi
Rawan Alotaibi
Fatimah Albahrani
Hala Aldossary
Haneen Albalawi
Haya Alhajri

CONTENT



Abstract

Why TESLA?

Key Challenges

Data Preparation and Wrangling

Statistical Analysis with NumPy

Filtering & Preparing with Pandas Library

Machine Learning Modeling

Data Visualization

ABSTRACT:

This project uses historical TESLA data to develop a predictive system that manages market risk and identifies opportunities. We moved from raw data cleaning to visualizing complex patterns “price and volume”. The final objective is to deliver a validated ML tool that provides crucial insights into how market prices might shift over time.

DATA SOURCE

We got our data from “kaggle”. This data set provides historical data of TESLA stock.

here is the data source link: TESLA Stock

Sample from the dataset

*Note that we manipulated the data for warnlling

Date	Open	High	Low	Close	Adj Close	Volume
29/06/10	3.8	5	3.508	4.778	4.778	93831500
30/06/10	5.158	6.084	4.66	4.766	4.766	85935500
01/07/10	5	5.184	4.054	4.392	4.392	41094000
02/07/10	4.6	4.62	3.742	3.84	3.84	25699000
06/07/10	4	4	3.166	3.222	3.222	34334500
07/07/10	3.28	3.326	2.996	3.16	3.16	34608500
08/07/10	3.228	3.504	3.114	3.492	3.492	38557000
09/07/10	3.516	3.58	3.31	3.48	3.48	20253000
09/07/10	3.516	3.58	3.31	3.48	3.48	20253000
09/07/10	3.516	3.58	3.31	3.48	3.48	20253000
09/07/10	3.516	3.58	3.31	3.48	3.48	20253000
15/07/10	3.988	4.3	3.8	3.978	3.978	18699000
09/07/10	3.516	3.58	3.31	3.48	3.48	20253000
19/07/10	4.274	4.45	4.184	4.382	4.382	12432500
21/07/10	4.132	4.18	3.9	4.044	4.044	6262500
22/07/10	4.1	4.25	4.074	4.2	4.2	4789000
26/07/10	4.3	4.3	4.06	4.19	4.19	4611000
27/07/10	4.182	4.236	4.052	4.11	4.11	3098500
29/07/10	4.154	4.176	4	4.07	4.07	3080000
30/07/10	4.04	4.088	3.91	3.988	3.988	2134500
02/08/10	4.1	4.194	4.066	4.184	4.184	3590500
03/08/10	4.2	4.39	4.164	4.39	4.39	6152500
04/08/10	4.39	4.436	4.17	4.252	4.252	4565000
05/08/10	4.308	4.31	4.01	4.09	4.09	3981000
06/08/10	4.02	4.032	3.904	3.918	3.918	3709500
09/08/10	3.98	3.996	3.89	3.92	3.92	4063500

FIRST 5 ROWS OF THE DATA

```
df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2010-06-29	3.800	5.000	3.508	4.778	4.778	93831500
1	2010-06-30	5.158	6.084	4.660	4.766	4.766	85935500
2	2010-07-01	5.000	5.184	4.054	4.392	4.392	41094000
3	2010-07-02	4.600	4.620	3.742	3.840	3.840	25699000
4	2010-07-06	4.000	4.000	3.166	3.222	3.222	34334500

WHY TESLA?

Extreme Volatility & High Returns

Brand Power and Market Influence

TESLA is a leader in the EV and technology sectors.

KEY CHALLENGES

1

Finding a well Historical Dataset

Our primary challenge was finding a complete and clean historical dataset spanning many years for a changing stock like Tesla.

2

Extreme Market Volatility

The stock's non linear behavior, especially after 2020, made accurate prediction highly complex and challenging for the ML model.

DATA SIZE

Here is an overview of the initial data size.

```
import pandas as pd
df = pd.read_csv('/content/TSLA.csv'
```

```
print("Data shape:", df.shape)
```

```
Data shape: (2930, 7)
```



DATA CLEANING AND WRANGLING

In this code we used panda library to find missing or duplicate values in the CSV file and count them.

```python
import pandas as pd
df = pd.read_csv('/content/TSLA.csv')

# Preview the rows
print(df)
# Data cleaning
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

# Reset index after cleaning
df = df.reset_index(drop=True)

print("Cleaned data shape:", df.shape)
```

```
[2930 rows x 7 columns]
Missing values:
Date          0
Open          1
High          1
Low           4
Close        16
Adj Close    16
Volume       18
dtype: int64

Duplicate rows  14
Cleaned data shape: (2930, 7)
```

DATA CLEANING AND WRANGLING

In this code we used panda library to find missing or duplicate values in the CSV file and count them.

```
# Remove duplicates
df = df.drop_duplicates()

# Remove rows with missing values
df = df.dropna()
# Reset index after cleaning
df = df.reset_index(drop=True)

print("Cleaned data shape:", df.shape)
```

```
Cleaned data shape: (2915, 7)
```

INITIAL STATISTICS WITH NUMPY

Here we have an initial statistics for the TESLA Stock prices and volume

```python
# Initial statistics
print("Open Price Stats:")
print(f"Mean: {np.mean(open_prices):.2f}")
print(f"Std: {np.std(open_prices):.2f}")
print(f"Min: {np.min(open_prices):.2f}")
print(f"Max: {np.max(open_prices):.2f}")

print("\nClose Price Stats:")
print(f"Mean: {np.mean(close_prices):.2f}")
print(f"Std: {np.std(close_prices):.2f}")
print(f"Min: {np.min(close_prices):.2f}")
print(f"Max: {np.max(close_prices):.2f}")

print("\nVolume Stats:")
print(f"Mean: {np.mean(volume):.0f}")
print(f"Std: {np.std(volume):.0f}")
print(f"Total Records: {len(df)}")
```

```
Open Price Stats:
Mean: 140.39
Std: 251.33
Min: 3.23
Max: 1234.41

Close Price Stats:
Mean: 140.46
Std: 251.41
Min: 3.16
Max: 1229.91

Volume Stats:
Mean: 31527950
Std: 28064568
Total Records: 2915
```

FILTERING , AND PREPARING FOR GRAPHS, ML WITH PANDAS LIBRARY

analysis and displaying the 4 quartiles, give max min values, check unique values and map a frequancy table for later ploting.

'summary statistics for numeric colums:'

	Open	High	Low	Close	Adj Close	Volume
count	2915.000000	2915.000000	2915.000000	2915.000000	2915.000000	2.915000e+03
mean	140.393743	143.511668	137.087177	140.463661	140.463661	3.152795e+07
std	251.371758	257.222061	245.066871	251.450992	251.450992	2.806938e+07
min	3.228000	3.326000	2.996000	3.160000	3.160000	5.925000e+05
25%	22.290999	23.702001	21.400999	22.557000	22.557000	1.343515e+07
50%	46.950001	47.782001	46.099998	46.958000	46.958000	2.513300e+07
75%	68.377003	69.487999	67.121997	68.336998	68.336998	3.997500e+07
max	1234.410034	1243.489990	1217.000000	1229.910034	1229.910034	3.046940e+08

Mean values:

	0
Open	1.403937e+02
High	1.435117e+02
Low	1.370872e+02
Close	1.404637e+02
Adj Close	1.404637e+02
Volume	3.152795e+07

dtype: float64

Median values:

	0
Open	4.695000e+01
High	4.778200e+01
Low	4.610000e+01
Close	4.695800e+01
Adj Close	4.695800e+01
Volume	2.513300e+07

dtype: float64

Missing values for each column:

	0
Date	0
Open	0
High	0
Low	0
Close	0
Adj Close	0
Volume	0

dtype: int64

Standard deviation:

	0
Open	2.513718e+02
High	2.572221e+02
Low	2.450669e+02
Close	2.514510e+02
Adj Close	2.514510e+02
Volume	2.806938e+07

dtype: float64

Number of unique values per column:

	0
Date	2915
Open	2639
High	2633
Low	2645
Close	2734
Adj Close	2734
Volume	2885

dtype: int64

Data Type：Numeric

FILTERING , AND PREPARING FOR GRAPHS, ML WITH PANDAS LIBRARY

analysis and displaying the 4 quartiles, give max min values, check unique values and map a frequancy table for later ploting.

Freqauncy of each unique'Volume' value:

Volume	count
8572500	2
4938000	2
23475000	2
3372500	2
16331500	2
...	...
21268500	1
19111000	1
13650000	1
15416500	1
39181000	1

2885 rows × 1 columns

dtype: int64

Value count for Date column, sorted by date:

Date	count
2010-06-29	1
2010-06-30	1
2010-07-01	1
2010-07-02	1
2010-07-06	1
...	...
2022-03-18	1
2022-03-21	1
2022-03-22	1
2022-03-23	1
2022-03-24	1

2915 rows × 1 columns

dtype: int64

Value count for Volume column, sorted by value:

Volume	count
592500	1
705000	1
805500	1
814000	1
856000	1
...	...
199404000	1
222126200	1
235325000	1
242119000	1
304694000	1

2885 rows × 1 columns

dtype: int64

Number of missing values per column in Tesla dataset:

	0
Date	0
Open	0
High	0
Low	0
Close	0
Adj Close	0
Volume	0

dtype: int64

Data Type: Numeric

FILTERING , AND PREPARING FOR GRAPHS, ML WITH PANDAS LIBRARY

analysis and displaying the 4 quartiles, give max min values, check unique values and map a frequancy table for later ploting.

Minimum values:

	0
Open	3.228
High	3.326
Low	2.996
Close	3.160
Adj Close	3.160
Volume	592500.000

dtype: float64

Maximum values:

	0
Open	1.234410e+03
High	1.243490e+03
Low	1.217000e+03
Close	1.229910e+03
Adj Close	1.229910e+03
Volume	3.046940e+08

dtype: float64

Data types for each column:

	0
Date	object
Open	float64
High	float64
Low	float64
Close	float64
Adj Close	float64
Volume	int64

dtype: object

Data Type: Object, Numeric

GENERATING A DATA FRAME FROM FEATURES REPRESENITING X AND TARGET REPRESENTING Y

(this section will later be used for optimization and machine learning)

```
# Define the feature(s) and target
# using 'Open', 'High', 'Low', 'Volume' as features to predict 'Close' price
features = df[['Open', 'High', 'Low', 'Volume']]
target = df['Close']

# Print shapes to verify
print("Features shape:", features.shape)
print("Target shape:", target.shape)
```

```
Features shape: (2915, 4)
Target shape: (2915,)
```

For Different Prediction Methods, Evaluations, and Visualizations We Used

MACHINE LEARNING MODEL

Our project extensively depend on Machine Learning to analyze and predict Tesla's stock performance. We've applied:

Scikit - learn

used for preprocessing, feature selection, and classical ML algorithms.

Scipy

ptimization techniques. Calculate the predicted values by iterating through the rows of the eatures DataFrame

TensorFlow

handles deep learning tasks such as building and training neural networks, also to predict cost.

SCIKIT-LEARN

used for preprocessing, feature selection, and classical ML algorithms

```python
# Define the feature(s) and target
# using 'Open', 'High', 'Low', 'Volume' as features to predict 'Close' price
features = df[['Open', 'High', 'Low', 'Volume']]
target = df['Close']

model = LinearRegression()
model.fit(features, target)                    # training the model
y_pred = model.predict(features)          # to predict new stocks

mse = mean_squared_error(target, y_pred)
r2 = r2_score(target, y_pred)

print("Scikit-learn Results")
print("Slope (θ1):", model.coef_[0])
print("Intercept (θ0):", model.intercept_)
print("MSE:", round(mse, 3))
print("R²:", round(r2, 3))
```

```
Scikit-learn Results
Slope (θ1): -0.6718974337673267
Intercept (θ0): -0.030089119430527944
MSE: 16.695
R²: 1.0
```



SCIPY

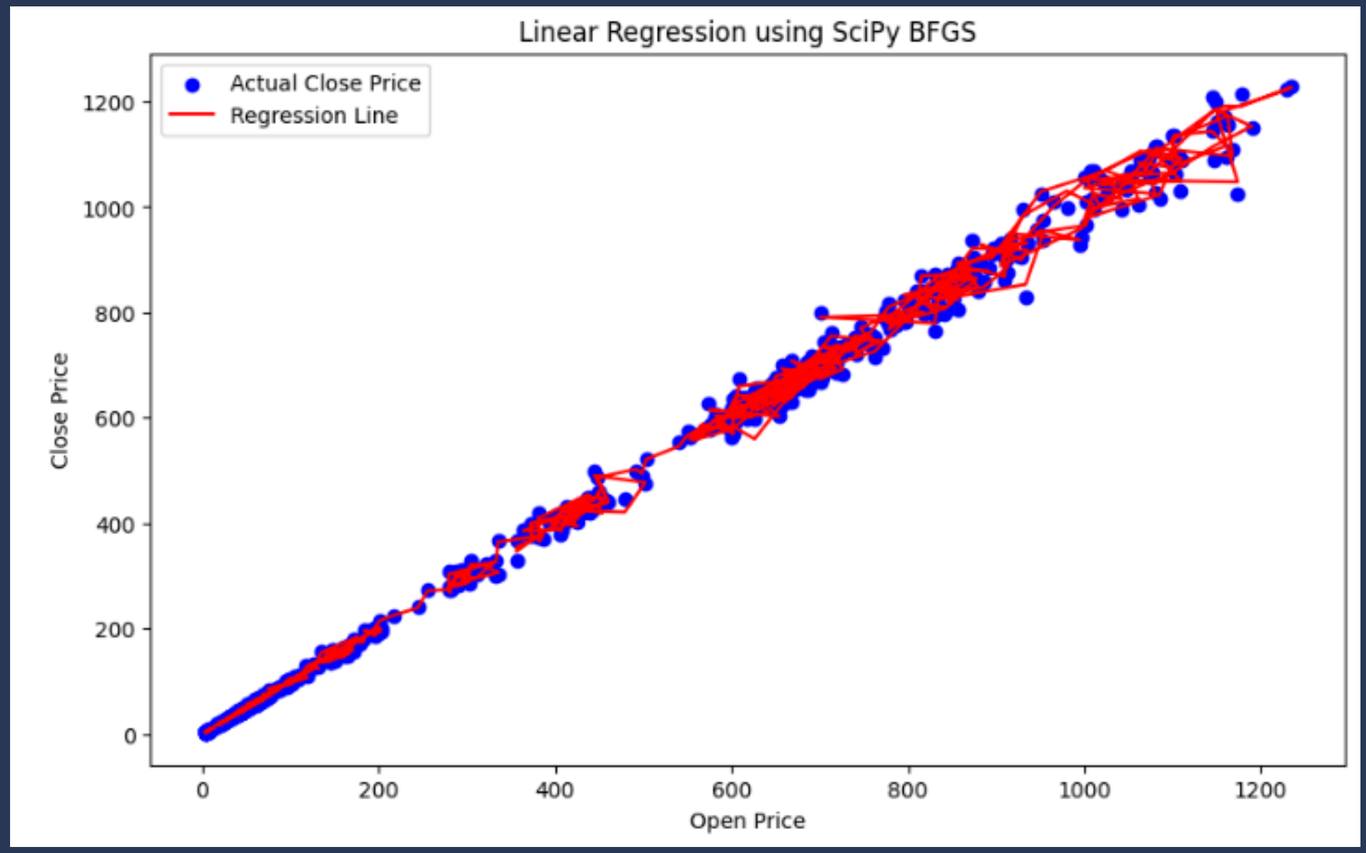
Optimization for the preivious midel.

```python
def cost_function(theta):
  theta0 ,theta1 = theta
    # Calculate the predicted values by iterating through the rows of the features DataFrame
    # Assuming 'Open' is the feature for simple linear regression
  y_pred = [theta0 + theta1 * features.iloc[i]['Open'] for i in range(len(features))]
  mse = sum((target[i] - y_pred[i])**2 for i in range(len(target))) / len(target)
  return mse

#using BGFS optimization for clean resualt
initial_guess = [0, 0]
result = optimize.minimize(cost_function, initial_guess, method='BFGS')

theta_optimized = result.x
print("\n SciPy Optimization Results")
print("θ0 (intercept):", round(theta_optimized[0], 3))
print("θ1 (slope):", round(theta_optimized[1], 3))
print("Final MSE:", round(result.fun, 3))
```

```
 SciPy Optimization Results
θ0 (intercept): 0.132
θ1 (slope): 1.0
Final MSE: 95.52
```



TENSORFLOW

Handles deep learning tasks such as building and training neural networks.

```
import tensorflow as tf #importing tensorflow
from sklearn.preprocessing import MinMaxScaler # Import MinMaxScaler
from sklearn.model_selection import train_test_split # Import train_test_split

print(tf.__version__)
#printing data
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(features) # Use 'features' instead of 'X'
#classfying data to test or train
X_train, X_test, y_train, y_test = train_test_split(X_scaled, target,
                                   test_size=0.2, random_state=42) # Use 'target' instead of 'y'
```

```
2.19.0
```

TENSORFLOW

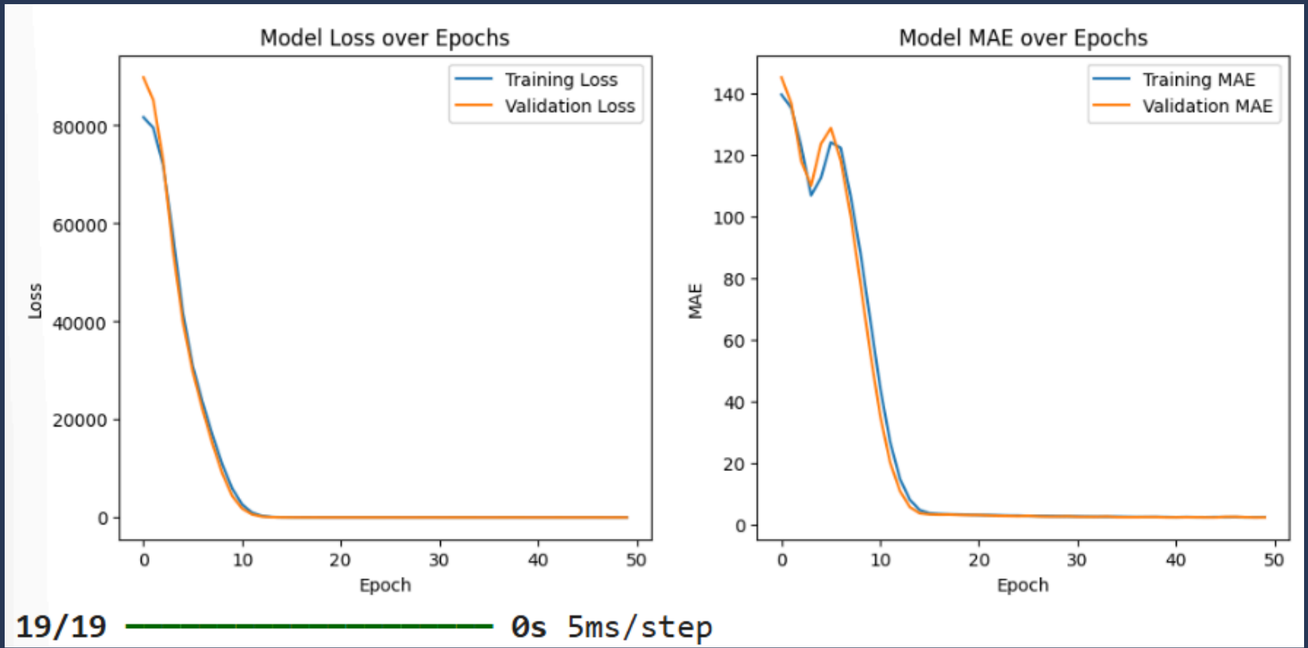
Code was written using Sequential to predict cost.

```python
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)  # give only 1 predction to cost
])
#this block to test the model after machine learning
model.compile(optimizer='adam', loss='mean_squared_error', metrics=['mae'])
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.1)
loss, mae = model.evaluate(X_test, y_test)
print("Test Loss:", loss)
print("Test MAE:", mae)
```

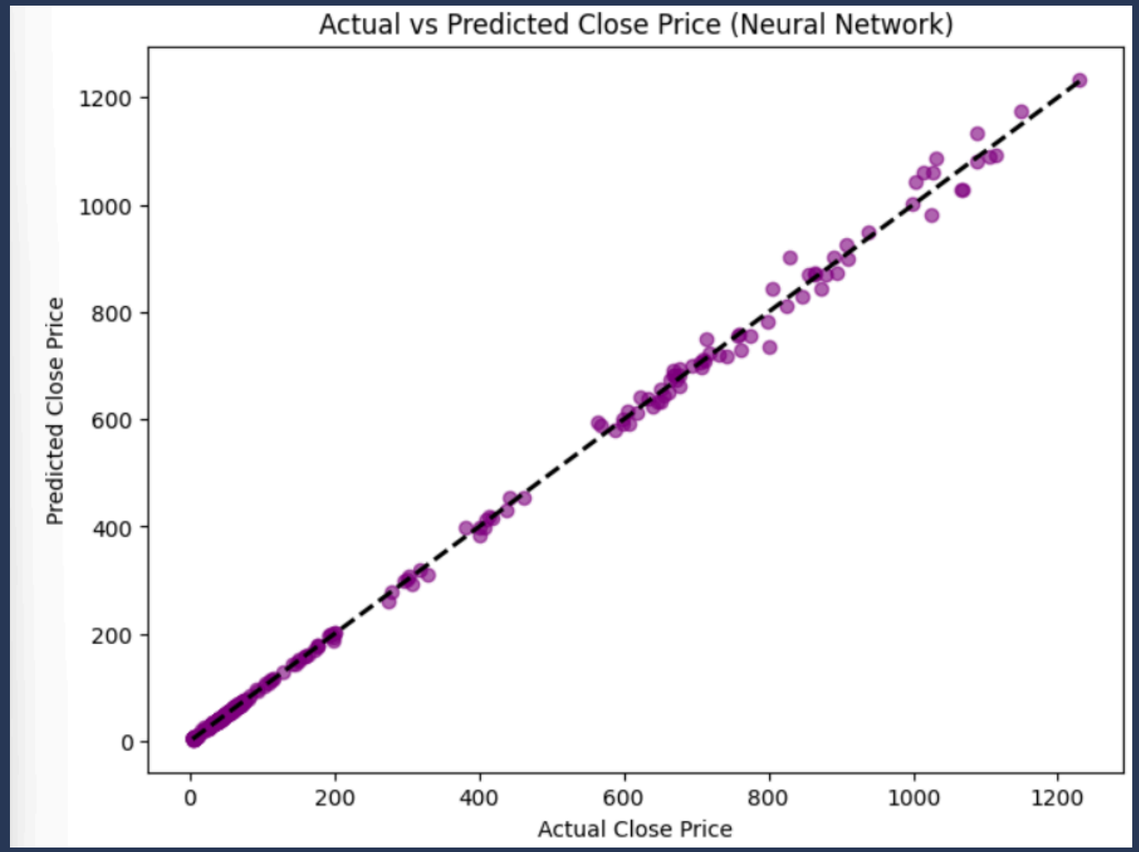
```
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 40.0890 - mae: 2.4080 - val_loss: 40.9108 - val_mae: 2.3569
Epoch 40/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 42.4164 - mae: 2.3651 - val_loss: 38.9129 - val_mae: 2.2649
Epoch 41/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 40.8084 - mae: 2.3761 - val_loss: 39.4207 - val_mae: 2.2815
Epoch 42/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 33.1599 - mae: 2.0960 - val_loss: 37.5857 - val_mae: 2.3127
Epoch 43/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 38.5090 - mae: 2.2596 - val_loss: 40.1914 - val_mae: 2.2986
Epoch 44/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 36.4130 - mae: 2.2467 - val_loss: 38.5553 - val_mae: 2.2605
Epoch 45/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 35.2967 - mae: 2.1969 - val_loss: 38.8698 - val_mae: 2.2398
Epoch 46/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 37.2249 - mae: 2.3306 - val_loss: 43.1110 - val_mae: 2.4274
Epoch 47/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 40.0527 - mae: 2.4276 - val_loss: 41.3324 - val_mae: 2.4295
Epoch 48/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 45.5994 - mae: 2.4164 - val_loss: 37.1996 - val_mae: 2.3112
Epoch 49/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 39.3195 - mae: 2.3761 - val_loss: 40.0423 - val_mae: 2.2275
Epoch 50/50
66/66 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 39.0974 - mae: 2.3432 - val_loss: 37.7723 - val_mae: 2.2462
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 57.8322 - mae: 2.5725
Test Loss: 71.11193084716797
Test MAE: 2.8995635509490967
```

TENSORFLOW

comparing the resualts with ploting




```
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step
```



VISUALIZING TESLA'S CLOSING PRICE

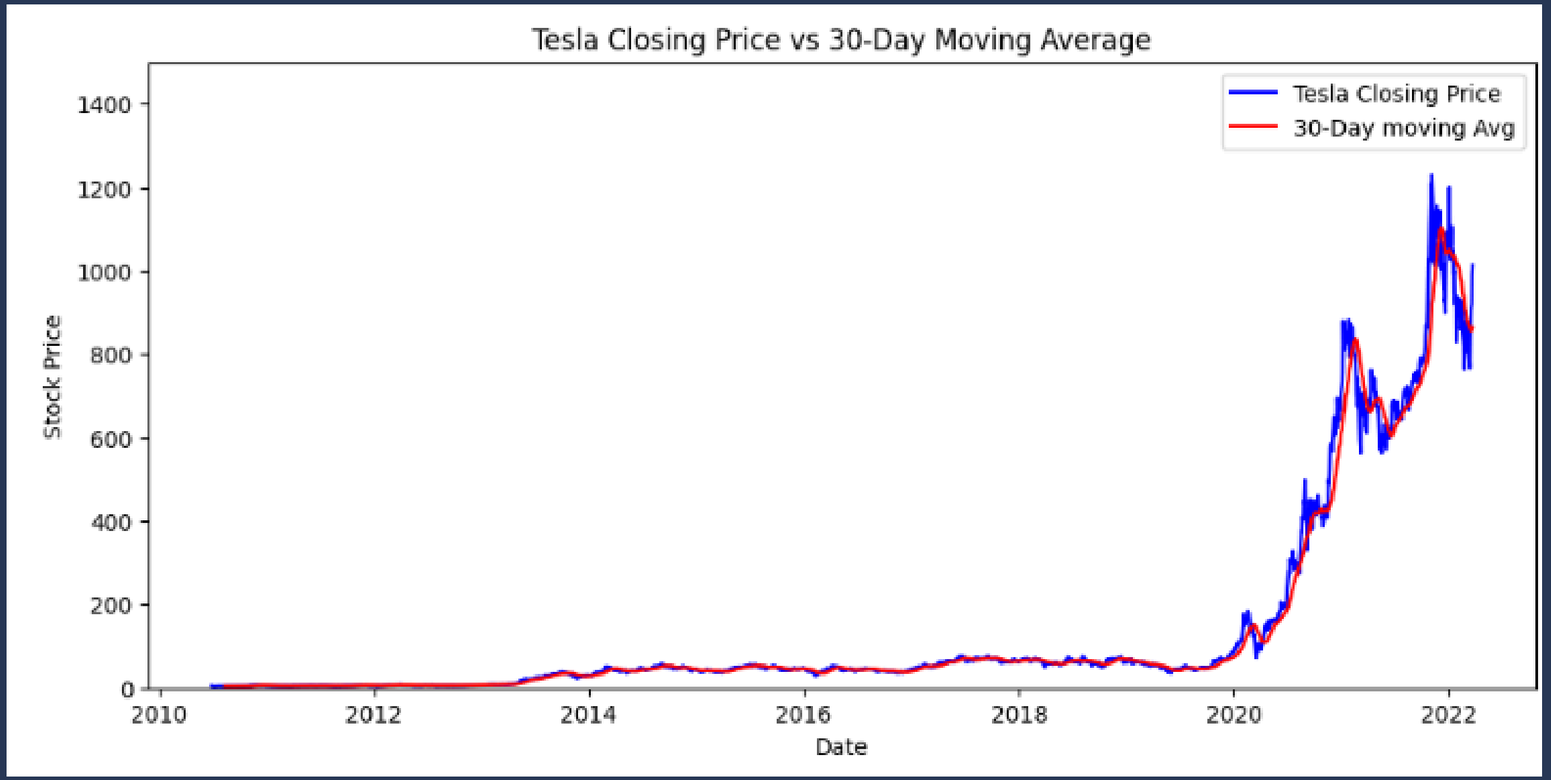
with a 30 day moving average to show both daily changes and the overall trend.

```
x = df['Date']
y = df['Close']

x_numeric = np.arange(len(x))

df['Data'] = pd.to_datetime(df['Data'])
df['30-Day moving Avg'] = df['Close'].rolling(30).mean()  # calculate 30-day moving average

plt.figure(figsize=(11,5))
# plot tesla stock price with moving average
plt.plot(df['Date'], df['Close'], label='Tesla Closing Price', color='blue') # plot actual closing price
plt.plot(df['Date'],df['30-Day moving Avg'], label='30-Day moving Avg', color='red')  # plot moving average line
plt.title('Tesla Closing Price vs 30-Day Moving Average')
plt.xlabel('Date')
plt.ylabel('Stock Price')
plt.ylim(0,1500)
plt.legend()
plt.show()
```



TESLA STOCK DATA DISTRIBUTION ANALYSIS

For Open, Close, and Volume to see their distributions and detect outliers

```
# create one wide figure with 3 subplots
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)# first subplot
plt.hist(open_prices, bins=30, color='darkcyan') # histogram for Open prices
plt.title('Open Prices')

plt.subplot(1,3,2)    # second subplot
plt.hist(close_prices, bins=30, color='hotpink') # histogram for Close prices
plt.title('Close Prices')

plt.subplot(1,3,3)    # third subplot
plt.hist(volume, bins=30, color='darkblue') # histogram for Volume
plt.title('Volume')

plt.tight_layout()
plt.show()


df_recent = df[(df['Date'] >= '2020-01-01') & (df['Date'] <= '2022-12-31')]

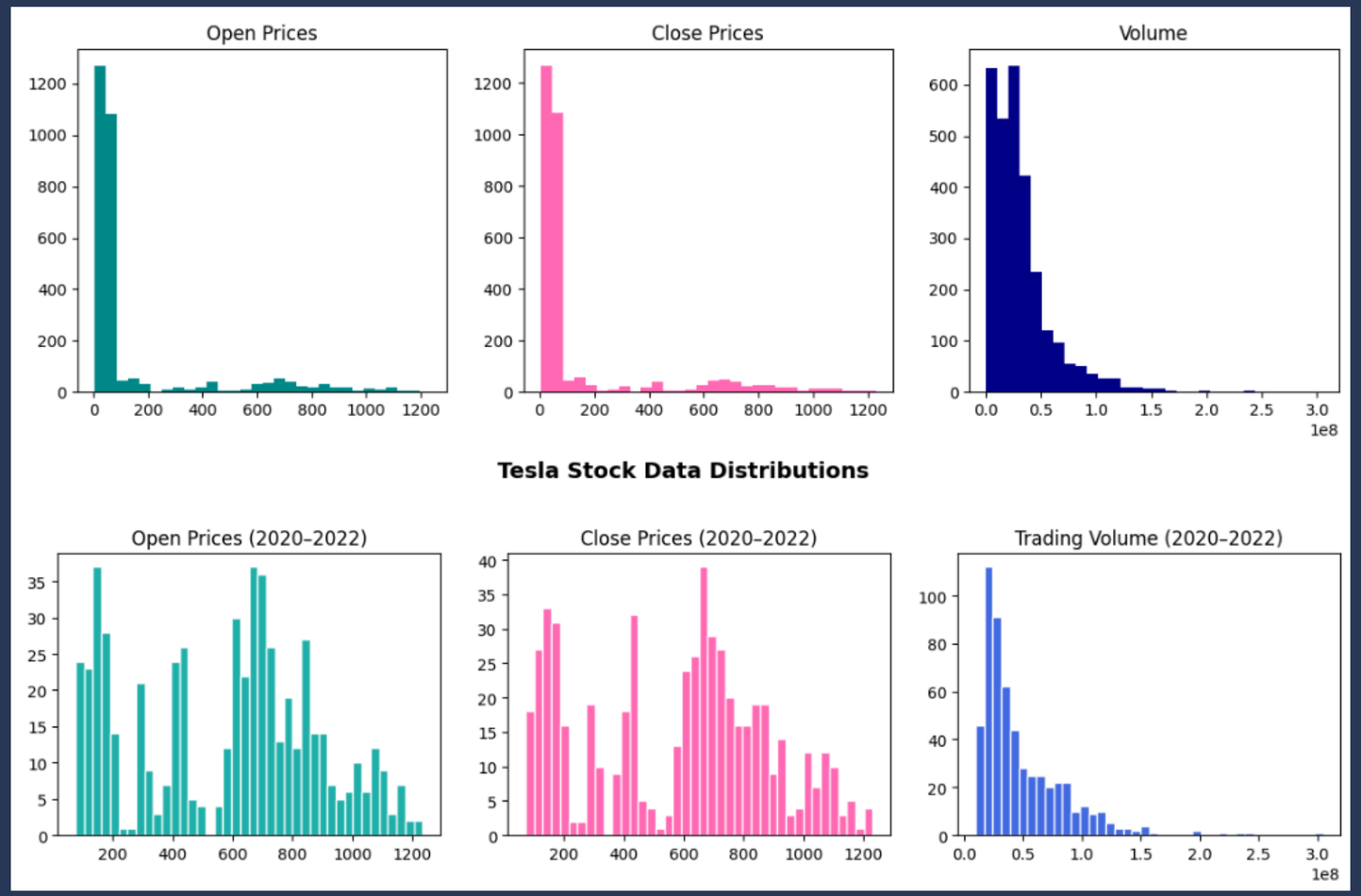
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.hist(df_recent['Open'], bins=40, color='#20B2AA', edgecolor='white')
plt.title('Open Prices (2020-2022)')

plt.subplot(1,3,2)
plt.hist(df_recent['Close'], bins=40, color='#FF69B4', edgecolor='white')
plt.title('Close Prices (2020-2022)')

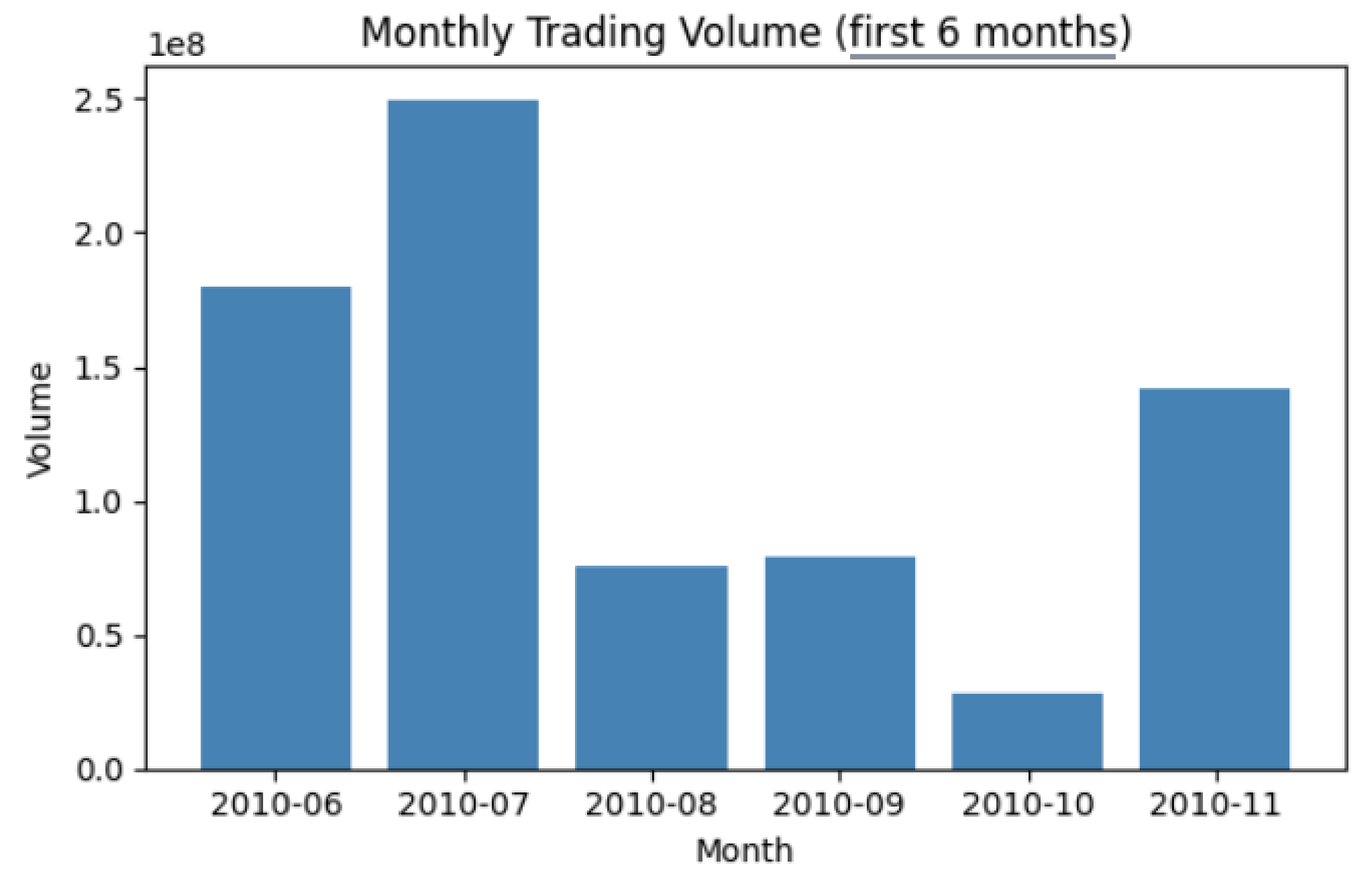
plt.subplot(1,3,3)
plt.hist(df_recent['Volume'], bins=40, color='#4169E1', edgecolor='white')
plt.title('Trading Volume (2020-2022)')

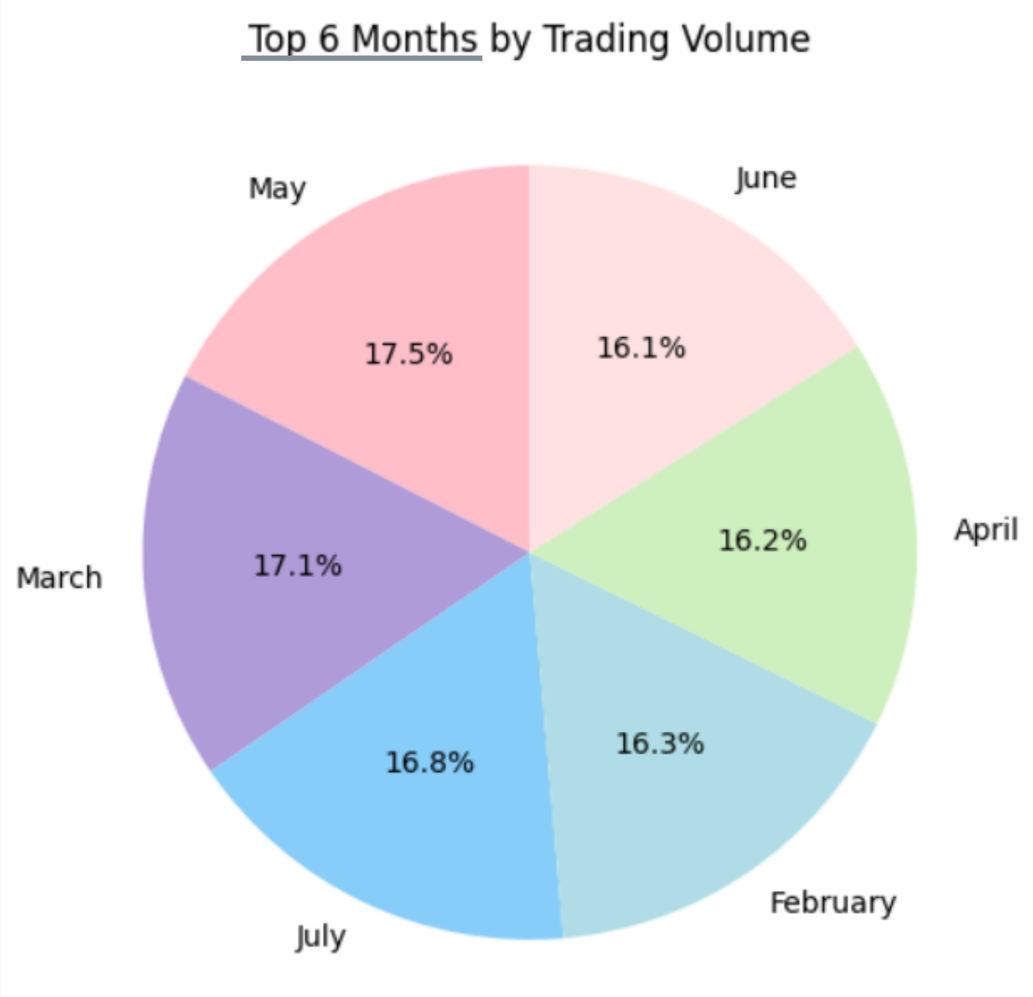
plt.suptitle('Tesla Stock Data Distributions', fontsize=14, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()
```



Market Activity

MOST ACTIVE MONTHS BY TRADING VOLUME



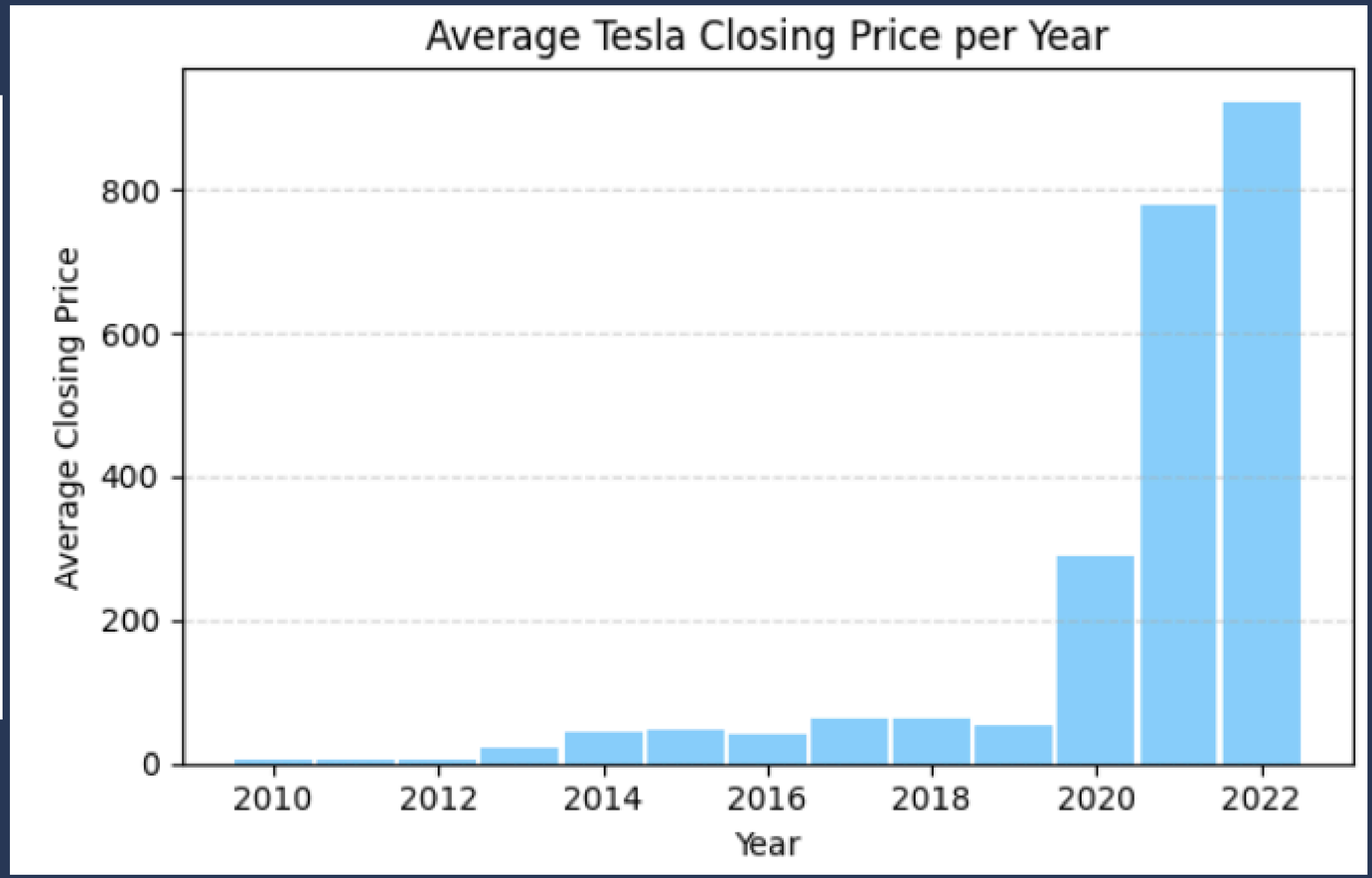


TESLA'S AVERAGE YEARLY CLOSING PRICES

This code cell calculates and visualizes the average closing price of Tesla stock for each year in the dataset.

```
# Average closing price per year (clean style)
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
yearly_avg = df.groupby('Year')['Close'].mean()

plt.figure(figsize=(6,4))
plt.bar(yearly_avg.index, yearly_avg.values, color='#87CEFA',
        width=0.94)
plt.title('Average Tesla Closing Price per Year')
plt.xlabel('Year')
plt.ylabel('Average Closing Price')
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



TESLA'S TRADING VOLUME VS CLOSING PRICE (2020-2022)

Each dot represents one trading day. We can notice that higher trading activity was often linked with stronger price movements

```
recent = df[(df['Date'] >= '2020-01-01') & (df['Date'] <= '2022-12-31')]

plt.figure(figsize=(7,5))

plt.scatter(recent['Volume'], recent['Close'], alpha=0.5, color='#0F766E',
            edgecolors='white', linewidths=0.3)
plt.title('Volume vs Closing Price (2020-2022)')
plt.xlabel('Volume')
plt.ylabel('Closing Price')

plt.grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```



THANK YOU!

Ready For Your Questions :)